

f1-mcp

Guía completa del proyecto, explicada con analogías
y todos los comandos de la terminal WSL

Servidor MCP que deja a Claude analizar carreras de Fórmula 1
con datos de OpenF1 y citar el reglamento de la FIA

Juan Lara · octubre de 2026

Repositorio: github.com/JuanEliasLaraSoto/f1-mcp

Contenido

1. La idea en una página
2. Las piezas básicas: qué es cada cosa (con analogías)
3. La historia del proyecto, fase a fase
 - 3.1 Preparar el entorno en WSL
 - 3.2 Primer servidor MCP y primeras tools
 - 3.3 Consistencia sobre residuos (journal 01)
 - 3.4 Caché SQLite y reintentos (journal 02)
 - 3.5 Resources y prompts (journal 03)
 - 3.6 Reorganizar el código (journal 04)
 - 3.7 Calidad: errores, ruff, mypy, cobertura, CI (journal 05)
 - 3.8 Todo en inglés (journal 06)
 - 3.9 Conectarlo a Claude de verdad (journal 07)
 - 3.10 CLAUDE.md y specs (journal 08)
 - 3.11 Despliegue en un VPS con Docker y HTTPS (journal 09)
 - 3.12 RAG del reglamento FIA (journals 10 y 11)
4. Chuleta: todos los comandos agrupados
5. Glosario rápido
6. Preguntas típicas de entrevista y cómo responderlas

Una aclaración honesta — No tengo acceso al historial real de tu terminal WSL. Los comandos de esta guía están reconstruidos a partir del propio repositorio (README, CLAUDE.md, docs/server-setup.md, specs, journal, workflows de CI y el historial de git). Si quieres la lista exacta de lo que tecleaste, en WSL ejecuta `history > ~/mis-comandos.txt` y la tendrás en un fichero. Los placeholders como `SERVER_IP` se dejan así a propósito: nunca metas IPs privadas, claves ni contraseñas en documentos.

1. La idea en una página

Un modelo de lenguaje como Claude sabe mucho de Fórmula 1, pero **no tiene los datos de cada vuelta** y, cuando hace cuentas de cabeza con cientos de números, se equivoca o se inventa cosas. f1-mcp le da a Claude un conjunto de **herramientas** para pedir datos reales de OpenF1 (una API gratuita con tiempos por vuelta, stints, paradas y resultados desde 2023) y, sobre todo, para **calcular**: ritmo limpio, duelo vuelta a vuelta, consistencia, degradación de neumáticos y estrategia de toda la parrilla.

Además, tiene un buscador sobre el **Reglamento Deportivo de la FIA 2026** (un RAG), para que cuando ocurra una penalización o un Safety Car, Claude pueda citar el artículo exacto en vez de responder de memoria.

Analogía — Piensa en Claude como **el piloto**: sabe conducir y razonar, pero desde el cockpit no ve la telemetría. f1-mcp es **el muro de boxes**: los ingenieros tienen las pantallas con todos los datos, hacen los cálculos y le pasan por radio un resumen claro ("vas 0,18 s por vuelta más rápido que Hamilton en las vueltas comparables"). Y cuando el piloto pregunta "¿por qué me han penalizado?", el ingeniero abre el reglamento por el artículo exacto. El protocolo MCP es **la radio**: el idioma estándar con el que piloto y muro se entienden.

Lo que lo diferencia de un simple "envoltorio" de API

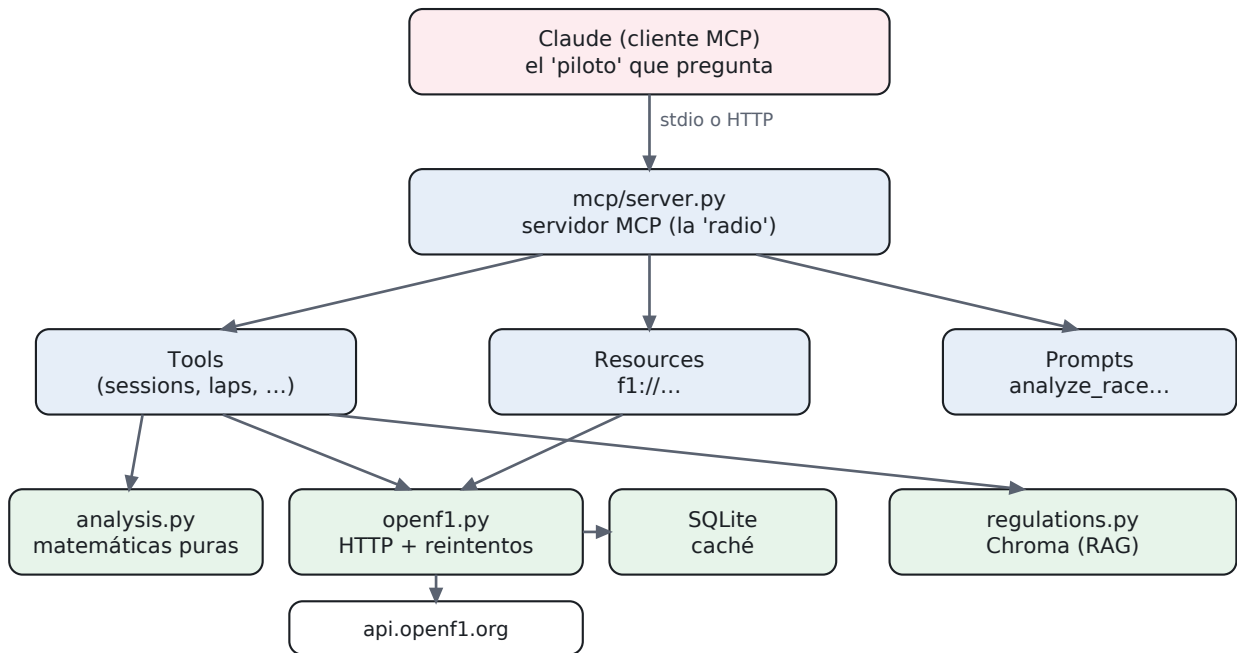
- **Calcula, no solo devuelve datos.** Cientos de filas crudas se convierten en unas pocas líneas que el modelo puede razonar.
- **Explica qué número es fiable.** Por ejemplo, el duelo vuelta a vuelta es más justo que la mediana porque compara vueltas con la misma carga de combustible.
- **Está construido como software profesional:** tests sin red, 95 % de cobertura, tipado estricto, CI/CD, Docker, HTTPS y un diario de decisiones (journal).

Ejemplo real (GP de Italia 2025, Monza)

```
compare_drivers(9912, 16, 44) # Leclerc vs Hamilton
```

Median pace: LEC is 0.335 s/lap faster than HAM.

Lap-by-lap duel (50 comparable laps): LEC faster in 34,
mean gap -0.179 s (negative = LEC faster).



Arquitectura: Claude habla con el servidor MCP; las tools usan las matemáticas puras (analysis.py), el cliente HTTP con caché (openf1.py) y el índice del reglamento (Chroma).

2. Las piezas básicas: qué es cada cosa

Antes de la historia, el vocabulario. Cada concepto con su analogía de F1 (o de la vida diaria) para que lo puedas explicar en una frase.

Concepto	Qué es	Analogía
WSL	Windows Subsystem for Linux: un Linux real (Ubuntu) dentro de Windows.	Un garaje Linux dentro de tu casa Windows: mismas herramientas que los servidores de verdad, sin salir de casa.
MCP	Model Context Protocol: estándar abierto para conectar modelos con herramientas y datos.	La radio del equipo, o un USB-C: un único enchufe que vale para cualquier cliente (Claude Code, Claude Desktop, otros).
Tool	Función que el modelo decide llamar con argumentos. Devuelve texto.	El piloto pide por radio: "¿cómo voy contra Hamilton?" y el ingeniero calcula.
Resource	Datos de solo lectura con una URI (f1://...). Devuelve JSON.	La carpeta de documentos que dejas sobre la mesa antes de la reunión.
Prompt	Plantilla que elige el usuario desde un menú.	El plan de carrera preparado de antemano: qué mirar y en qué orden.
OpenF1	API pública y gratuita con datos de F1 desde 2023.	El archivo de telemetría. Límite: 3 peticiones/s y 30/min.
uv	Gestor de Python ultrarrápido: versiones, entornos y dependencias.	El jefe de almacén que trae exactamente las piezas que pides.
uv.lock	Fichero con la versión exacta de cada dependencia.	La lista de piezas con número de serie: el coche se monta idéntico en cualquier fábrica.
pytest	Framework de tests.	El simulador: pruebas el coche sin salir a pista.
respx	Simula (mockea) las respuestas HTTP en los tests.	Un actor que hace de OpenF1 en el simulador: siempre responde lo mismo.
ruff	Linter y formateador.	El corrector ortográfico + la maquetación.
mypy (strict)	Comprueba los tipos antes de ejecutar.	El escrutinio técnico: revisa que cada pieza encaje antes de salir a pista.
pre-commit	Ejecuta ruff y mypy antes de cada commit.	El check de los mecánicos en el box antes de soltar el coche.
CI (GitHub Actions)	Tests y lint automáticos en cada push.	Los comisarios de la FIA: revisan todos los coches, siempre, sin excepciones.
CD	Despliegue automático si la CI pasa.	Si el coche pasa el escrutinio, sale a pista solo.
Docker	Empaqueta app + dependencias en una imagen que corre igual en todas partes.	Un contenedor de transporte: el coche viaja ya montado al circuito.
docker compose	Arranca varios contenedores juntos (f1-mcp + Caddy).	El camión del equipo con todo lo necesario.
VPS	Servidor Linux alquilado en internet (IONOS).	Alquilar un garaje en el circuito.

Concepto	Qué es	Analogía
Caddy	Servidor web que pone HTTPS automáticamente y reenvía al contenedor.	El recepcionista de la puerta: comprueba credenciales (certificado) y te pasa al garaje.
SSH + claves	Acceso remoto seguro con par de claves en vez de contraseña.	Una llave física frente a un código que se puede adivinar.
ufw	Cortafuegos de Ubuntu.	Una valla con solo tres puertas: 22, 80 y 443.
RAG	Retrieval-Augmented Generation: buscar textos relevantes y dárselos al modelo.	El bibliotecario que trae el artículo exacto del reglamento antes de que contestes.
Embedding	Vector numérico que representa el significado de un texto.	Coordenadas en un mapa del significado: textos parecidos quedan cerca.
Chroma	Base de datos vectorial.	El fichero de la biblioteca, ordenado por tema.

3. La historia del proyecto, fase a fase

El proyecto tiene 43 commits entre el 7 y el 10 de octubre de 2026. Cada fase: qué se hizo, por qué, la analogía y los comandos que se usan.

3.1 Preparar el entorno en WSL

Todo se desarrolla en Ubuntu dentro de WSL. Primero instalas uv, luego creas el proyecto con estructura de paquete (`src/f1_mcp/`) y fijas Python 3.12.

Analogía — Antes de construir el coche necesitas el taller: herramientas (uv), el chasis vacío (`uv init --package`) y decidir qué motor usas (Python 3.12).

Entorno inicial

```
# (en PowerShell de Windows, una sola vez) instalar WSL con Ubuntu
wsl --install

# --- ya dentro de Ubuntu (WSL) ---
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl

# Instalar uv (gestor de Python) y recargar la shell
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version

# Configurar git con tu identidad
git config --global user.name "Juan Lara"
git config --global user.email "juanelara03@gmail.com"

# Crear el proyecto como paquete (src layout) y fijar Python 3.12
uv init --package f1-mcp
cd f1-mcp
uv python pin 3.12
```

Las dependencias se añaden siempre con `uv add` (nunca con `pip`): así quedan registradas en `pyproject.toml` y bloqueadas en `uv.lock`.

Dependencias

```
# Dependencias de ejecución
uv add "mcp[cli]" httpx

# Dependencias de desarrollo (solo para tests y calidad)
uv add --dev pytest pytest-asyncio respx

# Instalar/sincronizar el entorno virtual (.venv) desde el lockfile
uv sync
```

Para la entrevista — ¿Por qué uv y no pip? Es mucho más rápido, gestiona también la versión de Python y el lockfile garantiza que tu PC, la CI y el Docker instalan exactamente las mismas versiones.

3.2 Primer servidor MCP y primeras tools

Commit `ee40c4c`: un servidor MCP con una sola tool, `list_sessions`, que devuelve las sesiones de un año con su `session_key` (el ID que necesitan todas las demás). Después llegaron `list_drivers`, `get_laps`, `compare_drivers` y `get_stints`, con un módulo `analysis.py` de funciones puras (sin red, sin ficheros: solo matemáticas).

Analogía — Separar **quién trae los datos** (`openf1.py`) de **quién hace las cuentas** (`analysis.py`) es como separar al mecánico que trae las piezas del ingeniero que hace los cálculos. El ingeniero puede practicar con datos inventados de propiedades conocidas (tests) sin esperar a que lleguen piezas reales.

El **docstring** de cada tool y sus tipos son literalmente lo que lee el modelo para decidir cuándo llamarla y con qué argumentos. Por eso se escriben con cuidado.

Vueltas limpias: se descartan la vuelta 1, las vueltas de salida de boxes, las vueltas sin tiempo y las que superan el 107 % de la mediana (Safety Car, tráfico, vueltas de entrada).

Ejecutar, probar y guardar

```
# Arrancar el servidor (transporte stdio: se queda esperando, Ctrl+C para salir)
uv run f1-mcp

# Probar el servidor sin Claude: un cliente MCP en memoria
uv run python scripts/try_client.py          # lista tools + schemas JSON
uv run python scripts/try_client.py list_sessions '{"year": 2025, "country": "Italy"}'
uv run python scripts/try_client.py list_drivers '{"session_key": 9912}'
uv run python scripts/try_client.py compare_drivers \
    '{"session_key": 9912, "driver_a": 16, "driver_b": 44}'
uv run python scripts/try_client.py get_stints \
    '{"session_key": 9912, "driver_number": 16}'

# Tests
uv run pytest
uv run pytest -v tests/test_analysis.py      # un fichero concreto, en detalle

# Guardar el trabajo
git init
git add .
git commit -m "Servidor MCP base con tool list_sessions y tests"
```

3.3 Consistencia sobre residuos (journal 01)

La primera versión medía la consistencia como la desviación típica de los tiempos limpios. En Monza salía ~1,1 s: absurdo para un piloto de F1. El motivo: los tiempos tienen **tendencia**. El coche gana ~0,055 s por vuelta al quemar combustible (~3 s en 53 vueltas) y cada stint tiene su propia degradación de neumáticos.

Solución:

- Para cada stint, ajustar una recta por mínimos cuadrados: `tiempo = a + b·vuelta`.
- Calcular el **residuo** de cada vuelta: cuánto se separa de su recta.
- Quitar outliers de forma robusta con la MAD ($|r - \text{mediana}| > 3\sigma$, con $\sigma = 1,4826 \cdot \text{MAD}$).
- Consistencia = desviación típica de los residuos que quedan.

Analogía — Imagina medir lo regular que es un ciclista que sube un puerto. Si comparas sus tiempos por kilómetro en bruto, verás que va más lento en las rampas duras, pero eso es la carretera, no el ciclista. Lo justo es comparar cada kilómetro con lo que **se esperaba** en ese tramo. En F1, la "carretera" es el combustible y los neumáticos; la recta del stint es lo esperado; el residuo es el piloto. Y la MAD es ignorar el kilómetro en el que se le cayó la cadena (la vuelta con tráfico).

Resultado real: $\sigma = 0,215$ s (LEC) y $0,256$ s (HAM), en lugar de $\sim 1,1$ s. Efecto secundario: pedir laps y stints de toda la sesión una vez y filtrar en local baja las peticiones a 3, dentro del límite de OpenF1.

Para la entrevista — Verificado con datos sintéticos: tendencia de $-0,1$ s/vuelta + ruido de $\pm 0,1$ s + una vuelta a $+5$ s. La desviación bruta sale $> 0,5$ s; la de residuos $< 0,2$ s, recuperando el ruido que se inyectó. Eso es un test con **propiedades conocidas**.

3.4 Caché SQLite y reintentos ante 429 (journal 02)

OpenF1 gratis permite 3 peticiones/segundo y 30/minuto. Una sola pregunta a Claude encadena varias tools, así que se superaba el límite. Solución:

- **Caché SQLite** en `~/.cache/f1-mcp/openf1.sqlite`. La clave es endpoint + parámetros ordenados. Las sesiones pasadas no cambian, así que no caduca. Las respuestas vacías **no** se guardan (puede que los datos aún no estén publicados).
- **sqlite3** de la librería estándar, sin ORM, y consultas parametrizadas (?) para evitar inyección SQL.
- **Reintento con backoff exponencial** ante un 429 ("demasiadas peticiones"): espera 1 s, luego 2 s; máximo 3 intentos.

Analogía — La caché es **la libreta del ingeniero**: si ya apuntó los tiempos de Monza 2025, no vuelve a llamar a la FIA para pedirlos. El backoff es como llamar a un restaurante que comunica: no remarcas como loco, esperas 1 segundo, luego 2... dando aire al otro lado.

Comandos útiles con la caché

```
# Ver la caché por dentro (opcional)
ls -lh ~/.cache/f1-mcp/
sqlite3 ~/.cache/f1-mcp/openf1.sqlite ".tables"

# Borrar la caché si quieres forzar datos frescos
rm ~/.cache/f1-mcp/openf1.sqlite
```

3.5 Resources y prompts (journal 03)

MCP tiene tres primitivas, y la regla para elegir es:

Primitiva	¿Quién la elige?	En f1-mcp
Tool	El modelo	compare_drivers, race_strategy, get_results... (devuelven texto)
Resource	El usuario / la app	f1://sessions/{year}, f1://session/{key}/results (JSON)

Primitiva	¿Quién la elige?	En f1-mcp
Prompt	El usuario, desde un menú	<code>analyze_race(year, circuit), driver_duel(...)</code>

También llegaron `get_results` (clasificación, gaps, DNF/DNS/DSQ), `race_strategy` (estrategia de toda la parrilla en una llamada) y el filtro por circuito en `list_sessions`. El prompt `driver_duel` obliga al modelo a separar ritmo y resultado: en Monza 2025 Norris acabó P2 tras una parada de 5,9 s frente a 1,9 s de Piastri.

Analogía — Tool = pedir algo al ingeniero. Resource = la hoja de resultados oficial que alguien te deja en la mesa. Prompt = el briefing pre-carrera estandarizado: "primero mira la parrilla, luego el ritmo, y no confundas ritmo con posición final".

Probar resources y prompts

```
uv run python scripts/try_client.py resources
uv run python scripts/try_client.py read f1://sessions/2025
uv run python scripts/try_client.py read f1://session/9912/results
uv run python scripts/try_client.py prompts
uv run python scripts/try_client.py prompt driver_duel \
    '{"year": 2025, "circuit": "Monza", "driver_a": "LEC", "driver_b": "HAM"}'
uv run python scripts/try_client.py race_strategy '{"session_key": 9912}'
```

3.6 Reorganizar el código por responsabilidad (journal 04)

`server.py` había pasado de 300 líneas mezclando todo. Se dividió en `config.py`, `formatting.py`, `analysis.py`, `mcp/server.py` y `mcp/tools/` (sessions, laps, strategy). Los ficheros se movieron con `git mv` para conservar su historial. Los mismos 19 tests pasaban antes y después: es un **refactor que preserva el comportamiento**.

Detalle técnico: `server.py` crea `mcp_server` y **al final** importa tools, resources y prompts para registrarlos. Así se evita una importación circular.

Analogía — Ordenar el garaje: llaves en un cajón, neumáticos en otro, ordenador de telemetría en su mesa. El coche no cambia, pero encuentras todo a la primera. Los tests son la prueba de que, tras ordenar, el coche sigue arrancando igual.

Refactor con historial conservado

```
mkdir -p src/f1_mcp/mcp/tools scripts
git mv probar_cliente.py scripts/probar_cliente.py
git mv src/f1_mcp/server.py src/f1_mcp/mcp/server.py
uv run pytest # mismos tests en verde antes y después
git status
git commit -am "Fase 5: reorganización por módulos"
```

3.7 Calidad: errores amables, ruff, mypy, cobertura y CI (journal 05)

- **Errores amables:** cualquier fallo de OpenF1 (timeout, sin red, 429 persistente, 5xx) se convierte en `OpenF1Error` con un mensaje legible, y el decorador `@friendly_errors` lo transforma en texto que el modelo puede explicar. Un decorador en lugar de un try/except en cada tool: el manejo de errores vive en un solo sitio. `functools.wraps` conserva nombre,

docstring y firma, así que el schema MCP no cambia.

- **ruff** (reglas E, F, I, UP, B) para lint y formato.
- **mypy strict**: pasó sin cambios porque el código ya estaba tipado. Genéricos PEP 695 (`[**P]`) para conservar la firma en el decorador.
- **pytest-cov** con mínimo del 80 %. Primera medida: 79 % (laps.py al 26 %). Tras añadir tests: **95 %**.
- **pre-commit**, **GitHub Actions** (tests + lint en cada push) y **Dependabot** (actualizaciones semanales de dependencias).

Analogía — Un error crudo (un stack trace) es como que la radio suelte un pitido de error. El decorador es el ingeniero que traduce: "no hay datos de esa sesión todavía, prueba con otra". Y pre-commit + CI son dos controles en serie: el de tu box (en tu PC) y el de los comisarios (en GitHub), que no se puede saltar.

Calidad y CI

```
uv add --dev pytest-cov mypy ruff pre-commit

uv run pytest --cov                # tests + cobertura (falla si < 80 %)
uv run ruff check . --fix          # lint, arreglando lo automático
uv run ruff format .              # formato
uv run mypy                       # tipos en modo estricto

uv run pre-commit install          # engancha los checks a cada git commit
uv run pre-commit run --all-files  # pasarlos todos a mano

# Subir a GitHub (primera vez) y ver la CI en la pestaña Actions
git remote add origin https://github.com/JuanEliasLaraSoto/f1-mcp.git
git branch -M main
git push -u origin main
```

3.8 Todo el código en inglés (journal 06)

Se tradujeron docstrings, comentarios, salidas de las tools, errores, nombres de prompts (`analizar_carrera` → `analyze_race`) y el cliente (`probar_cliente.py` → `try_client.py`). Motivo: es un proyecto de portfolio para audiencia internacional y, sobre todo, los docstrings **son lo que lee el modelo**; mezclar idiomas en su contexto no ayuda. Mismos 32 tests, 95 % de cobertura; las claves de caché no cambiaron.

```
git mv scripts/probar_cliente.py scripts/try_client.py
uv run pytest --cov && uv run ruff check . && uv run mypy
```

3.9 Conectado a Claude: primera conversación real (journal 07)

El servidor se registró en Claude Code dentro de WSL. Primer tropiezo: el PATH de WSL encontraba el binario de Claude Code de **Windows**, que intentaba lanzar el servidor en Windows y fallaba. Se solucionó usando la instalación de WSL (iniciando sesión desde Windows Terminal).

Analogía — Es como tener dos radios con el mismo nombre en el garaje: cogías la del vecino (Windows), que no está conectada a tu coche (WSL). **which -a claude** te enseña todas las radios que hay y cuál coges primero.

Registrar el servidor en Claude Code

```
# Comprobar qué 'claude' se está usando (debe ser el de Linux, no /mnt/c/...)
which -a claude

# Instalar Claude Code dentro de WSL e iniciar sesión
curl -fsSL https://claude.ai/install.sh | bash
claude                                # primera vez: login

# Registrar el servidor local (desde la carpeta del proyecto)
cd ~/f1-mcp
claude mcp add f1-mcp -- uv run --directory "$(pwd)" f1-mcp
claude mcp list                       # ver que aparece y está conectado
claude mcp get f1-mcp
# claude mcp remove f1-mcp           # por si hay que rehacerlo
```

Qué demostró la conversación: Claude llamó 6 veces a las tools por su cuenta (incluida la clasificación, sin pedírselo); la consistencia por residuos dio valores realistas; y el modelo eligió el duelo vuelta a vuelta (0,179 s/vuelta) frente a la mediana (0,335 s) porque la salida de la tool le explicaba por qué.

Grounding — Hallazgo importante: al final Claude añadió una penalización de Hamilton que **no venía de ninguna tool** ("si no recuerdo mal"). Es justo el tipo de afirmación sin respaldo (ungrounded) que hay que detectar con evals. Se quitó del ejemplo del README. Este hallazgo motiva más tarde el RAG del reglamento.

3.10 CLAUDE.md y desarrollo guiado por specs (journal 08)

CLAUDE.md es un fichero que Claude Code lee al empezar cada sesión: comandos, arquitectura, cómo añadir una tool y convenciones. Corto a propósito para no llenar el contexto. Además se adoptó **spec-driven development**: cada funcionalidad nueva empieza con una spec en **specs/** (objetivo, entradas, salidas, casos límite, tests de aceptación) que se aprueba antes de programar.

Analogía — CLAUDE.md es el **briefing** que recibe cada ingeniero nuevo al llegar al equipo. La spec es el **plano** de la pieza: nadie fabrica un alerón sin plano aprobado, y después el plano sirve para comprobar que la pieza fabricada (el código generado por IA) es la que se pidió.

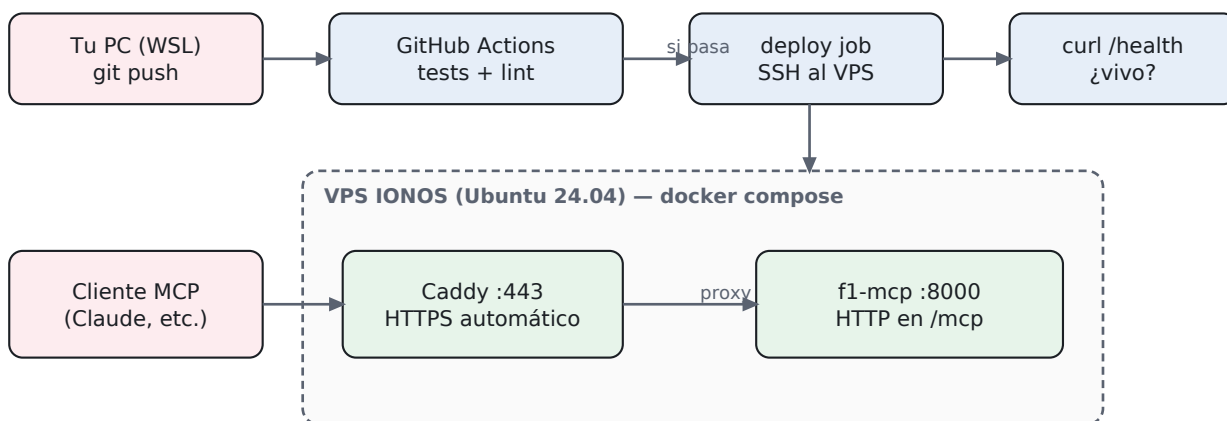
```
claude                                # dentro del proyecto: lee CLAUDE.md automáticamente
# /init                              # (dentro de Claude Code) genera un CLAUDE.md inicial
mkdir -p specs
```

3.11 Despliegue remoto: HTTP, Docker, VPS y HTTPS (journal 09)

Con stdio, f1-mcp solo funciona en tu máquina. Para que cualquiera lo use se necesita una URL pública. Lo que se hizo:

- **Transporte elegido por variable de entorno:** `F1_MCP_TRANSPORT=stdio|http`, `F1_MCP_HOST`, `F1_MCP_PORT`. En HTTP sirve MCP en `/mcp` (streamable HTTP, stateless) y un `GET /health` público.
- **Docker:** imagen con uv desde el lockfile. **Caddy** delante para HTTPS automático. La caché SQLite en un volumen con nombre (sobrevive a los redesplices).
- **VPS de IONOS** (Ubuntu 24.04): usuario `deploy` sin root, SSH solo con clave, cortafuegos con 22/80/443. Dominio gratis con **sslip.io** (la IP con guiones se convierte en un nombre de dominio).
- **Trampa encontrada:** escuchando en 127.0.0.1 el SDK activa protección contra DNS rebinding y devuelve 421 a otros Host; dentro del contenedor hay que escuchar en 0.0.0.0 (detrás de Caddy).
- **Despliegue continuo:** un job de GitHub Actions entra por SSH al VPS solo si tests y lint pasan, hace `git pull + docker compose up -d --build` y comprueba `/health`.

Analogía — stdio es el **intercomunicador interno** del garaje: solo funciona dentro. HTTP es **una línea telefónica pública**. Docker es el contenedor de transporte con el coche ya montado; Caddy es el recepcionista que comprueba la identidad (certificado HTTPS) y te pasa al garaje correcto; el usuario `deploy` es un mecánico con llave del garaje pero no de toda la fábrica (no root); ufw es la valla con tres puertas.



Flujo de despliegue: push → CI (tests + lint) → SSH al VPS → docker compose → /health.

En tu PC (WSL)

```

# Probar el modo HTTP en local (dos terminales)
F1_MCP_TRANSPORT=http uv run f1-mcp
curl http://127.0.0.1:8000/health # -> {"status":"ok"}

# Construir y arrancar con Docker en local
docker compose up -d --build
docker compose ps
docker compose logs -f f1-mcp
docker compose down
  
```

Montar el VPS una sola vez (docs/server-setup.md)

```
# 0. Crear tu par de claves SSH en WSL (si no lo tenías)
ssh-keygen -t ed25519 -C "juanelara03@gmail.com"

# 1. Acceso y actualizaciones (como root)
ssh-copy-id root@SERVER_IP
ssh root@SERVER_IP
apt update && apt upgrade -y

# 2. Docker
curl -fsSL https://get.docker.com | sh

# 3. Usuario 'deploy' sin contraseña, en el grupo docker, con tu clave
adduser --disabled-password --gecos "" deploy
usermod -aG docker deploy
mkdir -p /home/deploy/.ssh
cp /root/.ssh/authorized_keys /home/deploy/.ssh/authorized_keys
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh && chmod 600 /home/deploy/.ssh/authorized_keys

# 4. Cortafuegos: solo 22, 80 y 443
ufw allow OpenSSH && ufw allow 80/tcp && ufw allow 443/tcp
ufw --force enable
ufw status

# 5. Prohibir login con contraseña
sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
systemctl restart ssh

# 6. Arrancar f1-mcp (como deploy)
ssh deploy@SERVER_IP
git clone https://github.com/JuanEliasLaraSoto/f1-mcp.git && cd f1-mcp
echo "DOMAIN=SERVER-IP-WITH-DASHES.sslip.io" > .env
docker compose up -d --build
curl https://SERVER-IP-WITH-DASHES.sslip.io/health
```

Preparar el despliegue continuo (los secrets van en GitHub, nunca en el código)

```
# Clave dedicada SOLO para que GitHub Actions entre al VPS
ssh-keygen -t ed25519 -f ~/.ssh/f1mcp_deploy -N "" -C "github-actions-deploy"
ssh-copy-id -i ~/.ssh/f1mcp_deploy.pub deploy@SERVER_IP

# Huella del servidor, para el secret VPS_KNOWN_HOSTS
ssh-keyscan SERVER_IP

# Mostrar la clave privada para pegarla en el secret VPS_SSH_KEY
cat ~/.ssh/f1mcp_deploy
# GitHub > Settings > Secrets and variables > Actions:
#   VPS_SSH_KEY, VPS_KNOWN_HOSTS, VPS_USER (=deploy), VPS_HOST (=SERVER_IP)
```

Usar el servidor desplegado desde cualquier sitio

```
claude mcp add --transport http f1-mcp https://217-154-7-81.sslip.io/mcp
curl https://217-154-7-81.sslip.io/health
```

Para la entrevista — ¿Por qué VPS + Docker y no un PaaS? Más barato y demuestra Linux, Docker y CI/CD. Sin autenticación porque los datos son públicos; el riesgo asumido es que alguien abuse y agote el límite de OpenF1.

3.12 RAG sobre el Reglamento Deportivo FIA (journals 10 y 11)

OpenF1 dice **qué** pasó; el reglamento dice **qué norma** se aplica. Sin el reglamento, Claude responde "por qué" de memoria: el mismo fallo de datos inventados del journal 07. Nueva tool: `search_regulations(query, k)`. Solo **recupera** artículos; el modelo cliente redacta la respuesta y cita el número (el servidor no llama a ningún LLM).

El pipeline

Paso	Qué hace	Por qué
1. Extraer	<code>scripts/extract_regulations.py</code> lee el PDF con PyMuPDF	PyMuPDF solo hace falta en desarrollo
2. Limpiar	Quita cabeceras de página, el índice y los Apéndices B3-B5	B5 contiene cambios de 2027 con los mismos números de artículo
3. Trocear (chunking)	Un trozo por sub-artículo numerado (B5.13.2), con título de ruta (Safety Car (SC) > During a SC Deployment)	El reglamento ya viene en unidades autocontenidas y el número es la cita
4. Limitar tamaño	Trozos de más de 1.000 caracteres se parten por frases (#1, #2)	El modelo de embeddings solo lee ~256 piezas de palabra
5. Indexar	<code>f1-mcp-build-index</code> : all-MiniLM-L6-v2 (local, CPU) → Chroma, coseno	Sin API key, sin coste por consulta
6. Buscar	Consulta en inglés → top-k artículos con su similitud	El modelo traduce primero; el corpus es inglés

Analogía — El RAG es un **bibliotecario**. Primero corta el libro por artículos (chunking), no por páginas, porque la gente pide "el artículo del Safety Car", no "la página 47". Luego coloca cada artículo en un **mapa del significado** (embeddings): los que hablan de adelantamientos bajo Safety Car quedan juntos. Cuando preguntas, sitúa tu pregunta en el mismo mapa y te trae los artículos más cercanos. El bibliotecario no contesta por ti: te da los textos y tú (Claude) redactas la respuesta citando.

Lecciones del PDF real (journal 11)

El primer chunker se escribió contra fragmentos de una versión antigua y se probó con texto sintético. Con el PDF real (Issue 09) aparecieron problemas que los tests no veían: número, título y texto en líneas separadas (títulos vacíos), el índice colado como trozos, cabeceras de página pegadas al texto, el Apéndice B5 con texto de 2027, la FIA numerando dos artículos como B1.5.11, y un trozo de 16.000 caracteres. Tras corregirlo: de 323 a 459 trozos, todos con título y ninguno de más de 1.000 caracteres.

Lección — "Prueba un parser con el documento real cuanto antes: los datos sintéticos solo comprueban las suposiciones que ya tenías." Esta frase, del journal 11, es oro para una entrevista.

Comandos del RAG

```
uv add chromadb
uv add --dev pymupdf

# PDF de la FIA -> data/regulations_chunks.json (se sube a git: diff legible)
uv run python scripts/extract_regulations.py data/sporting.pdf

# JSON -> índice Chroma en data/chroma/ (descarga el modelo ~80 MB una vez)
uv run f1-mcp-build-index

# Probar la búsqueda
uv run python scripts/try_client.py search_regulations \
    '{"query": "overtaking under the Safety Car"}'
uv run python scripts/try_client.py search_regulations \
    '{"query": "unsafe release in the pit lane", "k": 3}'
```

Ejemplo: "overtaking under the Safety Car" devuelve B5.13.4 *Order of Cars Behind the SC* (similitud 0,71), B5.13.2 *During a SC Deployment* (0,67) y B5.13.5 *Duration of SC Period* (0,65). En el Dockerfile, el índice se construye al crear la imagen, así que el modelo de embeddings ya viaja dentro y el servidor nunca lo descarga en producción.

Pendiente (honestidad): solo consultas en inglés; un artículo largo puede ocupar varios huecos del top-k; y aún no hay un conjunto de evaluación etiquetado (siguiente paso: medir recall@k y comparar con búsqueda híbrida BM25 + vectores).

4. Chuleta: todos los comandos agrupados

Todos los comandos de la guía en un solo sitio, por tema. Ejecútalos desde la carpeta del proyecto (`cd ~/f1-mcp`) salvo que se indique lo contrario.

Terminal WSL básica

```
pwd                # ¿dónde estoy?
ls -la             # listar ficheros (también ocultos)
cd ~/f1-mcp        # ir al proyecto
cat README.md | less # leer un fichero (q para salir)
code .             # abrir VS Code desde WSL
explorer.exe .     # abrir la carpeta en el Explorador de Windows
history > ~/mis-comandos.txt # exportar tu historial real de comandos
```

uv y dependencias

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv init --package f1-mcp
uv python pin 3.12
uv add "mcp[cli]" httpx chromadb
uv add --dev pytest pytest-asyncio pytest-cov respx ruff mypy pre-commit pymupdf
uv sync            # instala todo desde uv.lock
uv sync --locked --all-groups # como la CI: falla si el lock no cuadra
uv lock --upgrade  # actualizar versiones del lock
uv run <comando>   # ejecutar dentro del entorno del proyecto
```

Ejecutar y probar el servidor

```
uv run f1-mcp                # servidor stdio
F1_MCP_TRANSPORT=http uv run f1-mcp # servidor HTTP en :8000
curl http://127.0.0.1:8000/health
uv run python scripts/try_client.py # tools y schemas
uv run python scripts/try_client.py TOOL '{json}' # llamar a una tool
uv run python scripts/try_client.py resources
uv run python scripts/try_client.py read f1://sessions/2025
uv run python scripts/try_client.py prompts
uv run python scripts/try_client.py prompt analyze_race \
    '{"year": 2025, "circuit": "Monza"}'
```

Calidad

```
uv run pytest            # tests
uv run pytest --cov      # tests + cobertura (mínimo 80 %)
uv run pytest -k strategy -v # solo tests cuyo nombre contenga 'strategy'
uv run ruff check . --fix  # lint
uv run ruff format .      # formato
uv run ruff format --check . # comprobar formato sin tocar (como la CI)
uv run mypy              # tipos estrictos
uv run pre-commit install
uv run pre-commit run --all-files # todo de una vez
```

Git y GitHub

```
git status          # qué ha cambiado
git diff            # ver los cambios
git add <archivo> / git add .
git commit -m "Mensaje claro en inglés"
git log --oneline   # historial resumido
git mv viejo.py nuevo.py    # mover conservando historial
git remote add origin https://github.com/JuanEliasLaraSoto/f1-mcp.git
git branch -M main
git push -u origin main
git pull            # traer cambios
git checkout -b feature/x    # rama nueva
```

Claude Code

```
which -a claude      # comprobar que usas el de WSL
claude               # abrir Claude Code en el proyecto
claude mcp add f1-mcp -- uv run --directory "$(pwd)" f1-mcp    # local (stdio)
claude mcp add --transport http f1-mcp https://217-154-7-81.sslip.io/mcp # remoto
claude mcp list
claude mcp get f1-mcp
claude mcp remove f1-mcp
```

Docker

```
docker compose up -d --build    # construir y arrancar en segundo plano
docker compose ps               # estado de los contenedores
docker compose logs -f f1-mcp   # ver logs en vivo (Ctrl+C para salir)
docker compose restart f1-mcp
docker compose down             # parar
docker image prune -f           # borrar imágenes viejas
docker volume ls                # volúmenes (f1-cache, caddy-data)
```

Servidor (VPS) y SSH

```
ssh-keygen -t ed25519 -C "tu-email"
ssh-copy-id deploy@SERVER_IP
ssh deploy@SERVER_IP
ssh-keyscan SERVER_IP
sudo ufw status
cd ~/f1-mcp && git pull --ff-only && docker compose up -d --build --remove-orphans
curl https://217-154-7-81.sslip.io/health
```

RAG del reglamento

```
uv run python scripts/extract_regulations.py data/sporting.pdf
uv run f1-mcp-build-index
uv run python scripts/try_client.py search_regulations '{"query": "..."}'
```

5. Glosario rápido

Término	En una frase
session_key	ID de una sesión en OpenF1 (Monza 2025 carrera = 9912, clasificación = 9908).
driver_number	Dorsal del piloto (16 = Leclerc, 44 = Hamilton).
Stint	Tramo de carrera con el mismo juego de neumáticos.
Vuelta limpia	Vuelta representativa: sin vuelta 1, ni salida de boxes, ni >107 % de la mediana.
Residuo	Diferencia entre el tiempo real y lo que predice la recta del stint.
MAD	Median Absolute Deviation: medida de dispersión robusta a valores extremos.
Degradación	Pendiente del tiempo por vuelta dentro del stint, corregida por combustible (~0,055 s/vuelta).
429	Código HTTP de "demasiadas peticiones" (rate limit).
Backoff exponencial	Esperar 1 s, 2 s, 4 s... entre reintentos.
Mock	Sustituto falso de algo externo (la API) en los tests.
Cobertura	Porcentaje de líneas de código ejecutadas por los tests.
Decorador	Función que envuelve a otra para añadirle comportamiento (@friendly_errors).
stdio	Transporte MCP por entrada/salida estándar: el cliente lanza el servidor como subproceso.
Streamable HTTP	Transporte MCP por HTTP, para servidores remotos.
Stateless	El servidor no guarda estado entre peticiones: cada una es independiente.
Reverse proxy	Servidor (Caddy) que recibe las peticiones y las reenvía al contenedor.
DNS rebinding	Ataque que la SDK mitiga rechazando Host desconocidos (el 421 que apareció).
Chunk	Trozo de texto indexado (aquí, un sub-artículo del reglamento).
Similitud coseno	Lo parecidos que son dos embeddings (1 = idénticos).
top-k	Los k resultados más parecidos.
Grounding	Que cada afirmación del modelo esté respaldada por datos de las tools.
Eval	Conjunto de pruebas que mide si el modelo usa bien las tools y no inventa.
Journal	Diario de decisiones numerado en journal/ (NN-titulo.md).
Spec	Documento previo con objetivo, entradas, salidas, casos límite y aceptación.

6. Preguntas típicas de entrevista

P: ¿Por qué MCP y no una API REST normal?

R: MCP es el estándar para que cualquier cliente de IA descubra y use herramientas: el modelo ve la descripción y el schema de cada tool y decide cuándo llamarla. Con REST tendría que programar la integración en cada cliente.

P: ¿Por qué calcular en el servidor y no dejar que el modelo haga las cuentas?

R: Los LLM no son fiables haciendo aritmética con cientos de números, y meter todas las filas en el contexto es caro. Las matemáticas viven en funciones puras y testeadas.

P: ¿Cómo mides la consistencia de un piloto?

R: Desviación típica de los residuos tras ajustar una recta por stint y quitar outliers con la MAD. La desviación bruta mide combustible y neumáticos, no al piloto (1,1 s frente a 0,2 s).

P: ¿Cómo gestionas el límite de OpenF1?

R: Caché SQLite permanente (los datos pasados no cambian), sin cachear respuestas vacías; reintentos con backoff exponencial ante 429; y pedir datos de toda la sesión una vez en lugar de por piloto.

P: ¿Cómo testear sin depender de la red?

R: respx simula OpenF1, cada test tiene una caché temporal vacía, y Chroma usa una función de embeddings falsa. También hay tests a través de una sesión MCP real en memoria.

P: ¿Por qué Chroma si con NumPy bastaba?

R: Con unos cientos de trozos, NumPy valdría. Elegí Chroma para trabajar con una base vectorial real (persistencia, metadatos, reconstrucción). El coste es una dependencia pesada (onnxruntime), que asumí conscientemente.

P: ¿Por qué trocear por artículo y no por tamaño fijo?

R: El reglamento ya está dividido en unidades autocontenidas y el número de artículo es la cita. Solo parto los trozos que pasan de 1.000 caracteres porque el modelo de embeddings ignora lo que hay más allá de ~256 piezas de palabra.

P: ¿Qué harías después?

R: Un conjunto de evals: preguntas con el artículo esperado (recall@k) y con las tools esperadas, comprobando grounding; probar búsqueda híbrida BM25 + vectores; y autenticación o rate limiting si el servidor público recibiera abuso.

P: ¿Cómo se despliega?

R: Push a main → GitHub Actions ejecuta tests y lint → si pasan, entra por SSH al VPS con una clave dedicada, hace git pull y docker compose up --build, y comprueba /health. Caddy pone HTTPS automático; el VPS solo abre 22/80/443 y no acepta contraseñas.

Resumen de una línea para abrir cualquier entrevista: "f1-mcp es un servidor MCP que da a Claude herramientas para analizar carreras de F1 con datos reales de OpenF1 —calculando ritmo, consistencia y estrategia en el servidor— y un RAG sobre el reglamento de la FIA para que cite las normas en lugar de inventarlas; con tests sin red, CI/CD y despliegue en un VPS con Docker y HTTPS."